

解説 1 末尾の文字 (Last Letter)

情報オリンピック日本委員会

解説担当：戸高空

時間制限：2 sec / メモリ制限：1024 MB

出典：<https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t1-review.html>

解説

末尾の文字が `G` であるかどうかで場合分けを行う。末尾の文字が `G` である場合には文字列 `S` の末尾の文字を除く 1 文字目から `N - 1` 文字目までを出力する。これは C++ の場合、`string` 型の文字列 `S` について、`S.substr(0, N - 1)` で目的の部分文字列を取り出せる。

末尾の文字が `G` でない場合には文字列 `S` に続けて文字列 `G` を出力すればよい。

解説 2 絶対階差数列 (Sequence of Absolute Differences)

情報オリンピック日本委員会

解説担当：米山瑛士

時間制限：2 sec / メモリ制限：1024 MB

出典：<https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t2-review.html>

解説

`for` 文や `while` 文を用いてシミュレーションすれば良いです。計算量は、一回の数列の書き換えに $O(N)$ かかり、それを $N-1$ 回行うので、 $O(N^2)$ になります。

絶対値を取るのを忘れないように注意してください。

解説 3 鐘 (Bell)

情報オリンピック日本委員会

解説担当：山縣龍人

時間制限：2 sec / メモリ制限：1024 MB

出典：<https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t3-review.html>

解説

小課題 1

小課題 1 では、鐘が 1 つしかないため、座標 x で聞こえる鐘の音の強さは $\max\{K - |x - A_1|, 0\}$ である。

これをそれぞれの家の座標ごとに求め、出力すれば良い。時間計算量は $\Theta(M)$ である。

小課題 2

小課題 2 では、鐘が複数になるため、最も強く聞こえる鐘を探す必要がある。

$i = 1, 2, \dots, N$ のそれぞれについて $\max\{K - |x - A_i|, 0\}$ を求め、この最大値を取れば、座標 x で聞こえる鐘の音の強さが分かる。

これをそれぞれの家の座標ごとに求め、出力すれば良い。時間計算量は $\Theta(NM)$ である。

小課題 3

小課題 3 では、 $\Theta(NM)$ 時間の解法では間に合わないため、最も強く聞こえる鐘を効率的に探す必要がある。

ここで、鐘の音の強さの定義から、「最も強く聞こえる鐘」とは「最も近くにある鐘」であることが分かる。すなわち、それぞれの家ごとに、その家に最も近い鐘を見つければ良い。

これは、二分探索によって家ごとに $\Theta(\log N)$ 時間でできるから、全体で $\Theta(N + M \log N)$ 時間で解ける。

また、 B をソートして尺取り法によって最も近い鐘を見つける方法もあり、この場合は $\Theta(N + M \log M)$ 時間で解ける。

解説 4 コイン集め 2 (Coin Collecting 2)

情報オリンピック日本委員会

解説担当：米田優峻

出典：https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t4-review.html

解説

注意 この解説では、上から i 行目、左から j 列目に置かれているコインを、**コイン (i, j)** と呼ぶことにします。

小課題 1 (累計 2 点)

この小課題の制約は $H = 1, W = 1$ ですので、ゲームの流れとしては「葵が 1 行目を選び、凜が 1 列目を選ぶ」しかありません。

したがって、 $S_{1,1}$ が # のとき 1 0 が答えになり、 $S_{1,1}$ が . のとき 0 1 が答えとなります。あとは if 文などで適切に場合分けすると、正解が得られます。

小課題 2 (累計 10 点)

この小課題の制約は $H = 1$ ですので、葵は 1 行目を選ぶしかありません。したがって、葵が操作を行った後における、凜のあり得る動きを全探索して、凜の得点が最も高くなるものを調べれば答えが分かります。

例として、次のようなケースを考えてみましょう。

```
1 3
##.
```

このとき、葵が 1 行目を選んでコインの向きを反転させた後の盤面は、次のようになります。

```
..#
```

それでは、凜は何列目を反転させるのが最適なのでしょうか。

まず 1 列目を選ぶと、裏のコインの枚数が 1 枚になります。次に 2 列目を選ぶと、裏のコインの枚数が 1 枚になります。そして 3 列目を選ぶと、裏のコインの枚数が 2 枚になります。

したがって、凜は 3 列目を選ぶのが最適であり、答えは「葵が 1 枚、凜が 2 枚」となります。

小課題 3 (累計 19 点)

この小課題の制約は $W=1$ ですので、凛の動きは 1 通りしかありません。そのため、葵の動きを全探索して、葵の得点が最も高くなるようなものを選べば正解となります。

例として、次のようなケースを考えましょう。

```
3 1
#
.
#
```

もし葵が 1 行目を選べば、盤面は次のようになります。このとき、凛が 1 列目のコインをすべて反転させると、獲得できるコインの枚数は「葵が 2 枚、凛が 1 枚」となります。

```
.
.
#
```

もし葵が 2 行目を選べば、盤面は次のようになります。このとき、凛が 1 列目のコインをすべて反転させると、獲得できるコインの枚数は「葵が 0 枚、凛が 3 枚」となります。

```
#
#
#
```

もし葵が 3 行目を選べば、盤面は次のようになります。このとき、凛が 1 列目のコインをすべて反転させると、獲得できるコインの枚数は「葵が 2 枚、凛が 1 枚」となります。

```
#
.
.
```

したがって、葵は 1 行目または 3 行目を選ぶのが最適であり、答えは「葵が 2 枚、凛が 1 枚」となります。

小課題 4 (累計 33 点)

この小課題は $H=2, W=2$ であるため、あり得る入力は全部で 16 通りしかありません。ですので、すべての場合の答えを手で計算して、if 文で場合分けすれば正解となります。

小課題 5/6 (累計 74 点)

ここからは少し難しいです。まずは例として、以下のようなケースを考えましょう。葵と凛は、どのように動くのが最適なのでしょうか。

```
3 3
##.
.##
#..
```

葵が 1 行目を選んだ場合

もし先手の葵が 1 行目を選べば、盤面は次のようになります。

```
..#
.##
#..
```

後手の凜は **表面のコインの枚数が最も多い列を反転させるのが最適** ですので、3 列目を選びます。すると盤面は次のようになり、獲得できるコインの枚数は「葵が 3 枚、凜が 6 枚」となります。

```
...
.#.
#.#
```

葵が 2 行目を選んだ場合

もし先手の葵が 2 行目を選べば、盤面は次のようになります。

```
##.
#..
#..
```

後手の凜は表面のコインの枚数が最も多い列を反転させるのが最適ですので、1 列目を選びます。すると盤面は次のようになり、獲得できるコインの枚数は「葵が 1 枚、凜が 8 枚」となります。

```
.#.
...
...
```

葵が 3 行目を選んだ場合

もし先手の葵が 3 行目を選べば、盤面は次のようになります。

```
##.
.##
.##
```

後手の凜は表面のコインの枚数が最も多い列を反転させるのが最適ですので、2 列目を選びます。すると盤面は次のようになり、獲得できるコインの枚数は「葵が 3 枚、凜が 6 枚」となります。

```
#..
..#
..#
```

解法のまとめ & 一般のケース

先程の例では、葵がそれぞれの行を選んだ場合に獲得できるコインの枚数は次のようになりました。したがって、葵は1行目または3行目を選ぶのが最適となり、答えは「葵が3枚、凛が6枚」となります。

- 葵が1行目を選ぶ：葵が3枚、凛が6枚
- 葵が2行目を選ぶ：葵が1枚、凛が8枚
- 葵が3行目を選ぶ：葵が3枚、凛が6枚

一般のケースでも同じように、（凛が最適に動く、すなわち表面のコインの枚数が最も多い列を選ぶという前提のもと）で葵の選ぶ行を全探索すれば、答えが求まります。

最後にプログラムの計算量を評価しましょう。葵の打てる手は H 通り存在し、それぞれについてコインの枚数を数えるのに計算量 $O(HW)$ かかるので、全体の計算量は $O(H^2W)$ となります。この小課題の制約は $H, W \leq 300$ ですので、実行時間制限の2秒には余裕をもって間に合います。

なお、実装は少し難しいですが、次のように関数を活用したり、適宜コメントを入れたりするなどのコツを使うと、実装がしやすくなります。（JOIのホームページで上手く表示させる都合上、include部分を削除しています）

```
// 変数
int H, W;
char c[309][309];
char d[309][309];

// 葵のターン：行 px を反転させる
void FlipAOI(int px) {
    for (int i = 1; i <= W; i++) {
        if (c[px][i] == '.') c[px][i] = '#';
        else c[px][i] = '.';
    }
}

// 凛のターン：列 py を反転させる
void FlipRIN(int py) {
    for (int i = 1; i <= H; i++) {
        if (c[i][py] == '.') c[i][py] = '#';
        else c[i][py] = '.';
    }
}

// 表になっているコインの枚数を数える
int CountAOICoins() {
    int cnt = 0;
    for (int i = 1; i <= H; i++) {
        for (int j = 1; j <= W; j++) {
            if (c[i][j] == '#') cnt++;
        }
    }
    return cnt;
}

// もし葵が行 px を選び、凛が最適に動いた場合、コインの枚数は何枚になるか？
int Simulation(int px) {
    // 葵のターンを行う
    FlipAOI(px);
```

```

// 凧がどの行を選ぶのが良いかを調べる
int Max_Omote = -1;
int Max_Index = -1;
for (int j = 1; j <= W; j++) {
    int sum = 0;
    for (int i = 1; i <= H; i++) {
        if (c[i][j] == '#') sum += 1;
    }
    if (Max_Omote < sum) {
        Max_Omote = sum;
        Max_Index = j;
    }
}

// 凧のターンを行う
FlipRIN(Max_Index);

// 枚数を返す
return CountA0ICoins();
}

int main() {
    // 入力
    cin >> H >> W;
    for (int i = 1; i <= H; i++) {
        for (int j = 1; j <= W; j++) cin >> c[i][j];
    }

    // 初期化用の配列をコピー
    for (int i = 1; i <= H; i++) {
        for (int j = 1; j <= W; j++) d[i][j] = c[i][j];
    }

    // 葵が選ぶ列を全探索
    int Answer = 0;
    for (int i = 1; i <= H; i++) {
        // 配列の初期化
        for (int j = 1; j <= H; j++) {
            for (int k = 1; k <= W; k++) c[j][k] = d[j][k];
        }
        // シミュレーション
        Answer = max(Answer, Simulation(i));
    }

    // 出力
    cout << Answer << " " << H * W - Answer << endl;
    return 0;
}

```

小課題 7 (累計 100 点)

先程の解法の計算量は $O(H^2W)$ でした。しかし、本小課題の制約は $H \times W \leq 500000$ であるため、 $H = 250000, W = 2$ のようなケースでは残念ながら TLE となってしまいます。

そこで、**各列における表のコインの枚数 `cnt[1], cnt[2], ..., cnt[W]` を前もって計算しておく** というアイデアを使うことで、アルゴリズムを高速化することができます。

例として、次のようなテストケースを考えましょう。

```

3 3
##.
.##
#..

```


このとき、初期状態における各列のコインの枚数は `cnt = [2, 2, 1]` となります。ここで、葵がそれぞれの行を選んだ場合のスコアをどうやって計算するかを実際に見ていきましょう。

葵が 1 行目を選んだ場合

まず、先手の葵が 1 行目を選んだ場合、コイン (1, 1) と (1, 2) が表から裏に変わり、コイン (1, 3) が裏から表に変わります。したがって、`cnt = [1, 1, 2]` に変化します。ここまでの計算処理は $O(W)$ 時間で行うことができます。

続いて、後手の凜は表のコインが最も多い列、すなわち `cnt[i]` が最も大きい列を選ぶのが最適ですので、3 列目を選びます。したがって、`cnt = [1, 1, 1]` に変化します。この計算処理も $O(W)$ 時間で行うことができます。

最後に、葵のコインの枚数は `cnt[i]` の合計値ですので、結果は「葵が 3 枚、凜が 6 枚」だと分かります。

葵が 2 行目を選んだ場合

まず、先手の葵が 2 行目を選んだ場合、コイン (2, 2) と (2, 3) が表から裏に変わり、コイン (2, 1) が裏から表に変わります。したがって、`cnt = [3, 1, 0]` に変化します。ここまでの計算処理は $O(W)$ 時間で行うことができます。

続いて、後手の凜は表のコインが最も多い列、すなわち `cnt[i]` が最も大きい列を選ぶのが最適ですので、1 列目を選びます。したがって、`cnt = [0, 1, 0]` に変化します。この計算処理も $O(W)$ 時間で行うことができます。

最後に、葵のコインの枚数は `cnt[i]` の合計値ですので、結果は「葵が 1 枚、凜が 8 枚」だと分かります。

葵が 3 行目を選んだ場合

まず、先手の葵が 3 行目を選んだ場合、コイン (3, 1) が表から裏に変わり、コイン (3, 2) と (3, 3) が裏から表に変わります。したがって、`cnt = [1, 3, 2]` に変化します。ここまでの計算処理は $O(W)$ 時間で行うことができます。

続いて、後手の凜は表のコインが最も多い列、すなわち `cnt[i]` が最も大きい列を選ぶのが最適ですので、2 列目を選びます。したがって、`cnt = [1, 0, 2]` に変化します。この計算処理も $O(W)$ 時間で行うことができます。

最後に、葵のコインの枚数は `cnt[i]` の合計値ですので、結果は「葵が 3 枚、凜が 6 枚」だと分かります。

解法のまとめ

このように、各列のコインの枚数 `cnt[i]` を記録するというアイデアを使うと、葵が各行を選んだときの最終的な結果を $O(W)$ 時間で求めることができます。葵の選択肢は H 通りあるので、プログラム全体の計算量は $O(HW)$ まで削減されます。

本問題の制約は $H, W \leq 500000$ と大きいですが、競技プログラミングでは 1 秒間に 1 億回以上の計算ができることが多いので、実行時間制限には余裕をもって間に合います。

最後に、C++ のサンプルコードは以下ようになります。（JOI のホームページで上手く表示させる都合上、include 部分を削除しています）

```
// 変数
int H, W;
int cnt[500009];
int cnt_init[500009];
string c[500009];

// もし葵が行 px を選び、凜が最適に動いた場合、コインの枚数は何枚になるか?
int Simulation(int px) {
    // 葵のターンを行う
    for (int i = 0; i < W; i++) {
```

```

        if (c[px][i] == '#') cnt[i] -= 1;
        if (c[px][i] == '.') cnt[i] += 1;
    }

    // 凜がどの行を選ぶのが良いかを調べる
    int Max_Omote = -1;
    int Max_Index = -1;
    for (int j = 0; j < W; j++) {
        if (Max_Omote < cnt[j]) {
            Max_Omote = cnt[j];
            Max_Index = j;
        }
    }

    // 凜のターンを行う
    cnt[Max_Index] = H - cnt[Max_Index];

    // 枚数を返す
    int Return = 0;
    for (int j = 0; j < W; j++) Return += cnt[j];
    return Return;
}

int main() {
    // 入力
    cin >> H >> W;
    for (int i = 0; i < H; i++) cin >> c[i];

    // cnt の値を計算する (cnt_init は初期化用の配列)
    for (int j = 0; j < W; j++) {
        cnt[j] = 0;
        for (int i = 0; i < H; i++) {
            if (c[i][j] == '#') cnt[j] += 1;
        }
        cnt_init[j] = cnt[j];
    }

    // 葵が選ぶ列を全探索
    int Answer = 0;
    for (int i = 0; i < H; i++) {
        // 配列の初期化
        for (int j = 0; j < W; j++) cnt[j] = cnt_init[j];
        // シミュレーション
        Answer = max(Answer, Simulation(i));
    }

    // 出力
    cout << Answer << " " << H * W - Answer << endl;
    return 0;
}

```

想定誤答例

最後に、よくある誤答例として、次のようなものがあります。（つまり一手読みです）

- 葵は、葵のターンが終わった後の表のコインの枚数が最大になるような手を打つ
- 凜は、凜のターンが終わった後の裏のコインの枚数が最大になるような手を打つ

この解法は、小課題 1・2・4 の制約を満たすテストケースにはすべて通りますが、入力例 5（以下を参照）のようなケースで間違った答えを出力してしまいます。

```

5 5
.###.
...##
..##.
.##..
##...

```

このケースでは葵が1行目を選択するのが最適であり、このとき葵は11枚のコインを獲得できます。しかし、先程の解法では2～5行目のいずれかを選択することになり、凜が最適な動きをすると9枚のコインしか獲得することができません。以下に、2行目を選択した場合の例を示します。

```

.###. .###. .#. #.
...## ###.. ##...
..##. -> ..##. -> ...#.
.##.. .##.. .#...
##... ##... ###..

```

解説 5 運河 (Canal)

情報オリンピック日本委員会

解説担当：菅井遼明

時間制限：2 sec / メモリ制限：1024 MB

出典：<https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t5-review.html>

解説

小課題 1

小課題 1 では $H = 1$ が成り立っています。このとき、JOIG 王国は標高が等しいマスからなる縦の長さが 1 の長方形に分かれます。

長方形の個数が 2 個以上である場合は、隣接する長方形の境界に運河を建設することで、平地の個数を長方形の個数と同じにすることができます。

長方形の個数が 1 個である、つまりすべてのマスの標高が等しい場合は、どのように運河を建設しても平地の個数は 2 個になります。

小課題 2

標高が異なる隣接するマスの境界でマス目を区切ると、マス目は何個かの部分に分かれます。これらの部分を連結成分と呼ぶことにします。また、連結成分に含まれるマスの個数をその連結成分の大きさと呼ぶことにします。

$H = 2$ のとき、各連結成分の形は非常に限られています。上下を確認しながら左から右へ標高が同じマスを通っていくことで、JOIG 王国に含まれるマスを連結成分に分解することができます。

また、運河を建設したとき、運河が連結成分の中を通過しているとき 2 個、そうでないとき 1 個の平地ができます。具体的には、連結成分の左端が左から L 列目、右端が左から R 列目であったとき、 $x = L, L + 1, \dots, R - 1$ のときは 2 個の平地が、そうでないときは 1 個の平地ができます。

累積和を用いることで、それぞれの x について平地の個数を高速に求めることができます。

時間計算量は $O(W)$ となります。

小課題 3

x としてありうる値を全探索します。運河の左側と右側それぞれについて、平地の個数を足し合わせればよいです。

平地の個数を調べるには、幅優先探索 (BFS) を行う、深さ優先探索 (DFS) を行う、素集合データ構造 (Union-Find 木, DSU) を用いるなどの方法があります。

時間計算量は $O(HW^2)$ となります。

小課題 4

$x = 1, 2, \dots, W - 1$ の順に運河の左側の平地の個数を調べることを考えます。各平地に含まれるマスの集合を管理する素集合データ構造を持ちます。

x を増やしたとき、 $x - 1$ 列目と x 列目の境界、 x 列目内の境界に対応する更新を行うことで、平地の個数を高速に調べることができます。

運河の右側の平地の個数についても、 $x = W - 1, W - 2, \dots, 1$ の順に同様に求めることができます。

素集合データ構造の実装により、時間計算量は $O(HW \alpha(HW))$ や $O(HW \log HW)$ となります。

解説 6 タイピング大会 (Typing Contest)

情報オリンピック日本委員会

解説担当：米田寛峻 (square1001)

出典：<https://www2.ioi-jp.org/joig/2022/2023-ho/2023-joig-ho-t6-review.html>

小課題 1 (累計 2 点)

この小課題では、文字列 S は $AAA\dots A$ の形をしているので、タイピング大会の参加者は同じキーを N 回打ち続けることになります。したがって、答えは $A_1 \times N$ ミリ秒であり、これを出力するプログラムを書けば小課題 1 に正解します。

```
#include <string>
#include <iostream>
using namespace std;

int main() {
    // 入力
    int N;
    string S;
    int Q;
    long long A, L, R;
    cin >> N >> S >> Q >> A >> L >> R;

    // 出力
    cout << A * N << endl;
    return 0;
}
```

注意点として、この問題の制約は $1 \leq N \leq 10^6$ および $1 \leq A_i \leq 10^{11}$ であるため、答えは最大で 10^{17} になります。そのため、例えば C++ で int 型を使った場合、int 型の整数が扱える範囲に収まりきらず「オーバーフロー」が起こります。その代わりに long long 型などを使いましょう。（※ Python の場合はあまり気にする必要はありません。）

小課題 2・3 (累計 9 点)

小課題 3 では、文字列 S は A, B, C の 3 種類からなります。ここで、 A, B, C のキーをひとつながりの位置に置けば、 D 以降のキーのことを考えずに「キーボードに A, B, C の 3 つのキーを配置する」と思っても大丈夫です。

このように考えると、キーボードの配置 $ABC, ACB, BAC, BCA, CAB, CBA$ の 6 通りについて、文字列 S を打つの何ミリ秒かかるかを求めれば、この最小値が答えになります。うまく実装するためには、以下のプログラムのように、キーボードの配置に対してかかる時間を計算する関数を作っておくとよいでしょう。

```

#include <string>
#include <iostream>
#include <algorithm>
using namespace std;

// キーボードの配置が与えられたとき、文字列 S を打つのにかかる時間を返す関数
long long calc(int N, string S, long long A, long long L,
               long long R, string keyboard) {
    long long cost = 0;
    for (int i = 0; i < N; i++) {
        if (i >= 1) {
            int prev_pos = keyboard.find(S[i - 1]);
            int next_pos = keyboard.find(S[i]);
            if (prev_pos < next_pos) {
                cost += (next_pos - prev_pos) * R;
            }
            if (prev_pos > next_pos) {
                cost += (prev_pos - next_pos) * L;
            }
        }
        cost += A;
    }
    return cost;
}

int main() {
    // 入力
    int N;
    string S;
    int Q;
    long long A, L, R;
    cin >> N >> S >> Q >> A >> L >> R;

    // 各キーボードの配置に対して、タイピングにかかる時間の計算
    long long CostABC = calc(N, S, A, L, R, "ABC");
    long long CostACB = calc(N, S, A, L, R, "ACB");
    long long CostBAC = calc(N, S, A, L, R, "BAC");
    long long CostBCA = calc(N, S, A, L, R, "BCA");
    long long CostCAB = calc(N, S, A, L, R, "CAB");
    long long CostCBA = calc(N, S, A, L, R, "CBA");

    // 出力
    cout << min({CostABC, CostACB, CostBAC, CostBCA, CostCAB, CostCBA}) << endl;
    return 0;
}

```

小課題 4 (累計 18 点)

小課題 4 では、文字が最大で 8 種類あります。S に含まれる文字の種類数を K とすると、キーボードの配置は $K! = 1 \times 2 \times \dots \times K$ 通りあります。例えば、K = 8 なら 40320 通りです。

小課題 3 と同様、これを全探索したいです。ここで、配列や文字列の並び替えを全探索するのは、C++ では `next_permutation`、Python では `itertools.permutations` を使うことでできます。

```

// C++ による実装 (小課題 4) : calc 関数は省略 (小課題 3 の実装の通り)
int main() {
    // 入力
    int N;

```

```

string S;
int Q;
long long A, L, R;
cin >> N >> S >> Q >> A >> L >> R;

// キーボードの配置を全探索
string keyboard = "ABCDEFGH";
long long answer = 9'000'000'000'000'000'000;
do {
    // keyboard = "ABCDEFGH", "ABCDEFGH", ..., "HGFEDCBA" の順に
    // 8! = 40320 通りそれぞれに対して処理が行われる
    long long cost = calc(N, S, A, L, R, keyboard);
    answer = min(answer, cost);
} while (next_permutation(keyboard.begin(), keyboard.end()));

// 出力
cout << answer << endl;
return 0;
}

```

```

# Python による実装 (小課題 4)
import itertools

# キーボードの配置が与えられたとき、文字列 s を打つのにかかる時間を返す関数
def calc(n, s, a, l, r, keyboard):
    pos = [keyboard.index(s[i]) for i in range(n)]
    cost = a
    for i in range(n - 1):
        if pos[i] < pos[i + 1]:
            cost += r * (pos[i + 1] - pos[i])
        if pos[i] > pos[i + 1]:
            cost += l * (pos[i] - pos[i + 1])
    cost += a
    return cost

# 入力
n = int(input())
s = input()
q = int(input())
a, l, r = map(int, input().split())

# キーボードの配置を全探索
answer = 10 ** 20
for keyboard in itertools.permutations('ABCDEFGH', 8):
    subanswer = calc(n, s, a, l, r, keyboard)
    answer = min(answer, subanswer)

# 出力
print(answer)

```

C++ や Python などの標準ライブラリに頼らずとも、再帰関数を使って同様の全探索を行うことができます。興味のある方はぜひインターネットなどで調べてみましょう。

小課題 3 の実装例にある `calc` 関数をそのまま使うと計算量は $O(K! \times K \times N)$ になりますが、どのキーがどの位置にあるかを前計算しておけば計算量 $O(K! \times N)$ になります。前者の計算量でも、C++ などの高速な言語であれば十分余裕をもって実行時間制限に間に合います。

小課題 5 (累計 32 点)

小課題 5 でも文字の種類数は最大 8 個ですが、 $N \leq 10^6$ なので小課題 4 の解法では TLE になってしまいます。何か工夫をして、キーボードの配置に対して文字列 S を打つのにかかる時間を高速に求められないでしょうか。

ここで、"同類のもの" をまとめて考えて効率化しましょう。 i 文字目のキーから $i+1$ 文字目のキーに移動するのにかかる時間は、ペア $(S[i], S[i+1])$ によって決まります。このペアとしてあり得るパターンは $K^2 \leq 64$ 通りしかありません。それぞれのペアが何カ所に現れるのかを前もって計算しておけば、各キーボードの配置に対して $O(K^2)$ で答えが求められます。

例えば、文字列 `ABCACCBBCACACABCAABCCCCCCCCA` をタイピングすることを考えます。このとき、各（前の文字, 次の文字）のペアの出現数は以下の表のようになります。

前\後	A	B	C
A	1	4	3
B	0	2	4
C	7	0	8

ここで、例えば $A=99, L=50, R=70$ 、キーボードの配置が `BAC` の場合：

- $A \rightarrow B$ に移動するのに 50 ミリ秒、 $B \rightarrow C$ に移動するのに 140 ミリ秒、 $C \rightarrow A$ に移動するのに 50 ミリ秒、 $A \rightarrow C$ に移動するのに 70 ミリ秒かかります。
- そのため、移動だけで $50 \times 4 + 140 \times 4 + 50 \times 7 + 70 \times 3 = 1320$ ミリ秒、と求められます。
- キーを打つのにかかる時間も含めると、 $1320 + 99 \times 30 = 4290$ ミリ秒、と求められます。

このように、文字列 S がいくら長かったとしても、（前の文字, 次の文字）のペアの出現数の表さえ作ってしまえば、各キーボードの配置に対してすぐに答えが求められます。

これを C++ で実装すると以下のようになります。計算量は $O(N + K! \times K^2)$ であり、小課題 5 に正解します。

```
#include <string>
#include <vector>
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    // 入力
    int N;
    string S;
    int Q;
    long long A, L, R;
    cin >> N >> S >> Q >> A >> L >> R;

    // 各（前の文字, 後の文字）の出現数を前計算する
    const int K = 8;
```

```

vector<vector<int>> table(K, vector<int>(K, 0));
for (int i = 0; i < N - 1; i++) {
    int prev_char = int(S[i] - 'A');
    int next_char = int(S[i + 1] - 'A');
    table[prev_char][next_char] += 1;
}

// キーボードの配置を全探索
vector<int> perm(K);
for (int i = 0; i < K; i++) perm[i] = i;

long long answer = 9'000'000'000'000'000'000;
do {
    // 例: perm = {3, 5, 1, 2, 4, 6, 7, 0} は配置 "DFBCEGHA" に対応
    long long cost = 0;
    for (int i = 0; i < K; i++) {
        for (int j = 0; j < K; j++) {
            // 「左から i 番目のキー」から「左から j 番目のキー」に移動するケース
            // 1 回あたりの時間 unit_cost を求める
            long long unit_cost = (i < j ? (j - i) * L : (i - j) * R);
            cost += table[perm[i]][perm[j]] * unit_cost;
        }
    }
    cost += A * N;
    answer = min(answer, cost);
} while (next_permutation(perm.begin(), perm.end()));

// 出力
cout << answer << endl;
return 0;
}

```

小課題 6 (累計 58 点)

この小課題を解くにあたって重要な考察は、左に移動する回数と、右に移動する回数の差が、最初に打つキーと最後に打つキーの位置関係だけで決まるということです。

例えば、下図では EACHBAG という文字列を、ABCDEFGHIJ と並んだキーボードで打つ例を示しています。ここでは、左に 11 回、右に 13 回移動しているので、右の方が 2 回多いです。なぜなら、最初に打つキー E よりも最後に打つキー G の方が 2 つ右にあるから、2 回多く右に移動しなければならないからです。



入力順: E → A → C → H → B → A → G

開始位置 E から終了位置 G までの正味移動は右へ 2 キー分

この性質を使って解いてみましょう。次の値を前計算しておくことを考えます。

- `left[d]` : 最初に打つキーの場所と最後に打つキーの場所の差が `d` であるとき、文字列 `S` を打つために最小で左に何回移動する必要があるか

ここで、 $\text{left}[-K+1], \text{left}[-K+2], \dots, \text{left}[K-1]$ を前計算しておけば、最終的な答えは次式の最小値で求められます。

$$N \times A + \text{left}[j] \times L + (\text{left}[j] + j) \times R$$

したがって、各参加者に対して $O(K)$ で答えを求められるようになりました。全体計算量は $O(N + K! \times K^2 + QK)$ なので、小課題 6 に正解します。

なお、小課題 6 の実装例は [こちらのリンク \(C++\)](#) に掲載されています。

小課題 7 (累計 81 点)

小課題 7 は動的計画法 (DP) で解くことができます。キーボードの配置を左から順に決めていくことを前提に、以下の値を計算していきます。

- $\text{dp}[T]$: 既にキーボードに配置した文字の集合が T であるときの、現時点での合計時間の最小値

DP の漸化式は次のようになります。ただし、 $c_{a\beta}$ を「文字列 S 中で、文字 a の次に文字 b が来る箇所の個数」とします（小課題 5 で作った表を思い出してください）。

$$\text{dp}[T] = \min_{a \in T} (\text{dp}[T \setminus \{a\}] + L \cdot \sum_{x \in T} \sum_{y \notin T} c_{y,x} + R \cdot \sum_{x \in T} \sum_{y \in T} c_{x,y})$$

少し式が難しいですが、後半の項は次の意味です。

- 左から $i+1$ 番目のキーから i 番目まで左に移動する回数が「 $x \in T, y \notin T$ となる (x, y) に対する $c_{y,x}$ の合計」である。
- 左から i 番目のキーから $i+1$ 番目まで右に移動する回数が「 $x \in T, y \in T$ となる (x, y) に対する $c_{x,y}$ の合計」である。

このため、消費時間にその分だけ足される、という意味です。なお、このように T のような集合を記録する DP は「ビット DP」と呼ばれ、 T の代わりに 2 進数の値を配列の添字に使います。興味のある方はインターネット等で調べてみましょう。

この漸化式にしたがった DP の計算には、計算量 $O(2^K \times K^2)$ がかかります。ただし、各 T に対して前計算も含めて考えると、全体計算量は $O(N + 2^K \times K^2)$ となり、小課題 7 に正解します。

(コードは準備中)

なお、 Q 人の参加者に対して計算する場合は計算量が $O(N + Q \times 2^K \times K^2)$ となり、一見正解しないと思われるが、各 T に対して次の 2 つの値を前計算することによって、計算量を $O(N + 2^K \times K^2 + Q \times 2^K \times K)$ に改善できます。

$$\bullet \sum_{x \in T} \sum_{y \notin T} c_{y,x}$$

$$\bullet \sum_{x \in T} \sum_{y \in T} c_{x,y}$$

そうすると、実装によっては小課題 6 にも正解できる場合があります。

小課題 8 (累計 88 点)

$L = R$ の場合は、合計移動回数を最小化することが、タイピングにかかる時間を最小化することになります。合計移動回数は、 $A = 0, L = 1, R = 1$ の場合を解けば求まります。

したがって、小課題 7 の解法をそのまま使って、全体計算量 $O(N + 2^K \times K + Q)$ で小課題 8 に正解することができます。

小課題 9 (累計 100 点)

小課題 6 の解法と小課題 7 の解法を組み合わせると、この問題の最後の小課題を解くことができます。

小課題 6 の解法のアイディアは、最初に打つ文字の位置と最後に打つ文字の位置によって、左に移動する回数と右に移動する回数の差が決まるということでした。つまり、次の値をすべての (a, b) に対して計算すれば、`left[-K+1], left[-K+2], ..., left[K-1]` の値が求まります。

- 文字 $S[1]$ のキーと文字 $S[N]$ のキーがそれぞれ左から a, b 番目のキーであるとき、タイピング中に左に移動する回数の最小値はいくつか？

上の問題の答えは、小課題 7 の DP で計算量 $O(2^K \times K)$ で求まります。ただし、小課題 7 の解説の最後に書かれているように、各 T に対して前述の 2 種類の和を前計算するものとします。

よって、`left[-K+1], left[-K+2], ..., left[K-1]` の値は、計算量 $O(2^K \times K^3)$ で求まります。この問題は全体計算量 $O(N + 2^K \times K^3 + QK)$ で解くことができ、満点を獲得することができます。

なお、実装例は [こちらのリンク \(C++\)](#) および [こちらのリンク \(Python\)](#) に掲載されています。